

Guided by: Dr. Sudha S

```
linkage_matrix = linkage(X, 'single')
plot = plt.figure(figsize=(10, 5))
dendrogram(linkage_matrix)
plt.title('Dendrogram for Iris Dataset using Single Linkage')
plt.xlabel('Data Points')
plt.ylabel('Distance')
plt.show()
```



```
model3 = AgglomerativeClustering(linkage = 'average', n_clusters = 3)
```

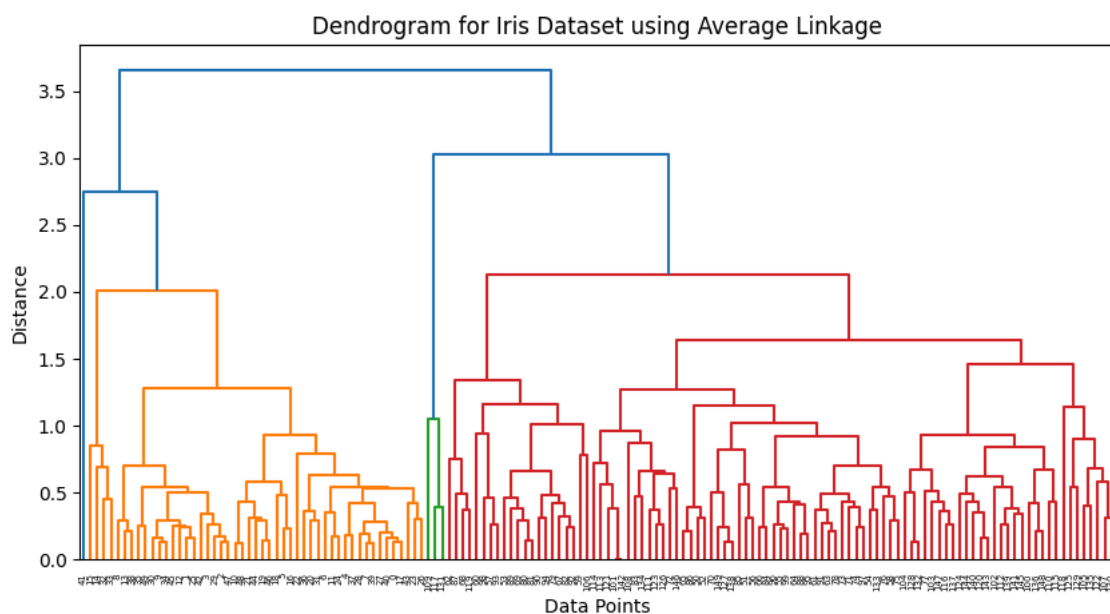
```
model3.fit(X)
```

```
AgglomerativeClustering
AgglomerativeClustering(linkage='average', n_clusters=3)
```

```
model3.labels_
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2,
       1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

```
linkage_matrix = linkage(X, 'average')
plot = plt.figure(figsize=(10, 5))
dendrogram(linkage_matrix)
plt.title('Dendrogram for Iris Dataset using Average Linkage')
plt.xlabel('Data Points')
plt.ylabel('Distance')
plt.show()
```



II. Let us consider data set mushroom have 22 baseline variables like cap-shape, capcolor, odor and so on were obtained for each of n = 8124 mushrooms, as well as the response of interest, a qualitative measure of category after baseline. Implement Principal Component Analysis for the given dataset. Perform the following tasks:

Tasks:

1. Data Loading and Exploration: Load the provided dataset. Explore the dataset to understand its characteristics, including the number of features and data points.
2. Data Preprocessing: Standardize or normalize the data if necessary.
3. Covariance Matrix: Calculate the covariance matrix of the dataset.
4. Eigenvalue and Eigenvector Computation: Compute the eigenvalues and eigenvectors of the covariance matrix.
5. Explained Variance: Calculate the explained variance for each principal component. Determine the cumulative explained variance to decide on the number of principal components to retain.
6. Dimensionality Reduction: Select the top 'k' principal components based on the cumulative explained variance. Transform the data into the reduced-dimensional space.
7. Visualization: Visualize the data in the reduced-dimensional space.
8. Interpretation: Discuss the results and interpret the impact of dimensionality reduction on the dataset.

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaricus-lepiota.data"
```

```
columns = [
    'class', 'cap-shape', 'cap-surface', 'cap-color', 'bruises', 'odor',
```

```
'gill-attachment', 'gill-spacing', 'gill-size', 'gill-color',
'stalk-shape', 'stalk-root', 'stalk-surface-above-ring',
'stalk-surface-below-ring', 'stalk-color-above-ring',
'stalk-color-below-ring', 'veil-type', 'veil-color', 'ring-number',
'ring-type', 'spore-print-color', 'population', 'habitat'
]
```

```
df = pd.read_csv(url, header=None, names=columns)
```

```
df.head()
```

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...	stalk-surface-below-ring	stalk-color-above-ring	stalk-color-below-ring	veil-type
0	p	x	s	n	t	p	f	c	n	k	...	s	w	w	p
1	e	x	s	y	t	a	f	c	b	k	...	s	w	w	p
2	e	b	s	w	t	l	f	c	b	n	...	s	w	w	p
3	p	x	y	w	t	p	f	c	n	n	...	s	w	w	p
4	e	x	s	g	f	n	f	w	b	k	...	s	w	w	p

```
y = df['class']
X = df.drop('class', axis=1)
```

```
label_encoders = {}
for col in X.columns:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
    label_encoders[col] = le
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
cov_matrix = np.cov(X_scaled.T)
print("\nCovariance matrix shape:", cov_matrix.shape)
```

```
Covariance matrix shape: (22, 22)
```

```
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[sorted_indices]
eigenvectors = eigenvectors[:, sorted_indices]

print("\nTop 5 eigenvalues:\n", eigenvalues[:5])
```

```
Top 5 eigenvalues:
[3.87410924 2.60339434 2.40073834 2.16674719 1.70303181]
```

```
explained_variance = eigenvalues / np.sum(eigenvalues)
cumulative_variance = np.cumsum(explained_variance)
```

```
for i, (ev, cv) in enumerate(zip(explained_variance, cumulative_variance)):
    print(f"PC{i+1}: Explained Variance = {ev:.4f}, Cumulative = {cv:.4f}")
```

```
PC1: Explained Variance = 0.1845, Cumulative = 0.1845
PC2: Explained Variance = 0.1240, Cumulative = 0.3084
PC3: Explained Variance = 0.1143, Cumulative = 0.4227
PC4: Explained Variance = 0.1032, Cumulative = 0.5259
PC5: Explained Variance = 0.0811, Cumulative = 0.6070
PC6: Explained Variance = 0.0664, Cumulative = 0.6734
PC7: Explained Variance = 0.0466, Cumulative = 0.7200
PC8: Explained Variance = 0.0453, Cumulative = 0.7653
PC9: Explained Variance = 0.0367, Cumulative = 0.8020
PC10: Explained Variance = 0.0339, Cumulative = 0.8359
PC11: Explained Variance = 0.0275, Cumulative = 0.8634
PC12: Explained Variance = 0.0258, Cumulative = 0.8892
PC13: Explained Variance = 0.0243, Cumulative = 0.9135
PC14: Explained Variance = 0.0227, Cumulative = 0.9362
PC15: Explained Variance = 0.0183, Cumulative = 0.9545
PC16: Explained Variance = 0.0148, Cumulative = 0.9693
```

```

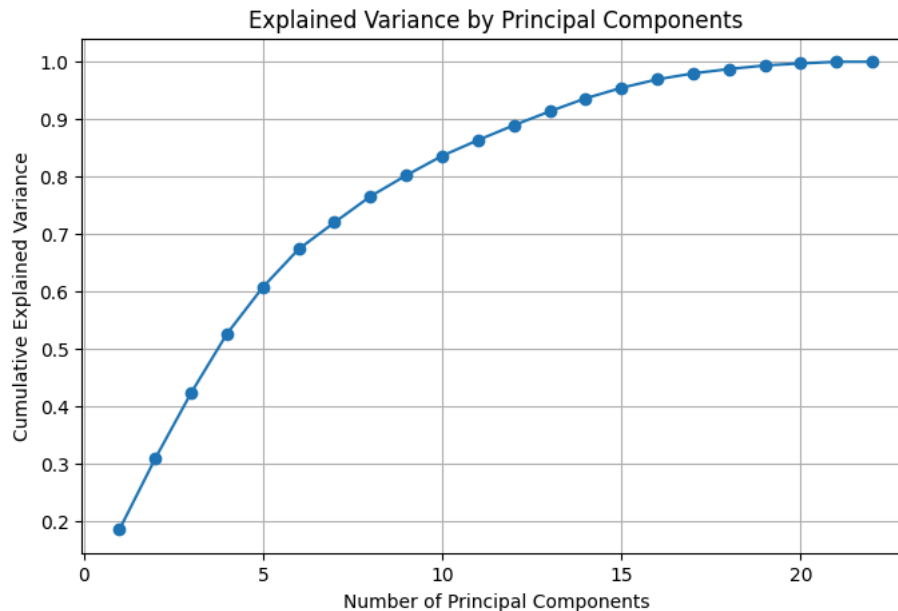
PC17: Explained Variance = 0.0104, Cumulative = 0.9797
PC18: Explained Variance = 0.0076, Cumulative = 0.9873
PC19: Explained Variance = 0.0063, Cumulative = 0.9936
PC20: Explained Variance = 0.0035, Cumulative = 0.9971
PC21: Explained Variance = 0.0029, Cumulative = 1.0000
PC22: Explained Variance = 0.0000, Cumulative = 1.0000

```

```

plt.figure(figsize=(8,5))
plt.plot(range(1, len(explained_variance)+1), cumulative_variance, marker='o')
plt.xlabel('Number of Principal Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Explained Variance by Principal Components')
plt.grid(True)
plt.show()

```



```

k = np.argmax(cumulative_variance >= 0.95) + 1
print(f"\nNumber of components to retain 95% variance: {k}")

```

Number of components to retain 95% variance: 15

```

W = eigenvectors[:, :k]
X_reduced = np.dot(X_scaled, W)
print("\nReduced data shape:", X_reduced.shape)

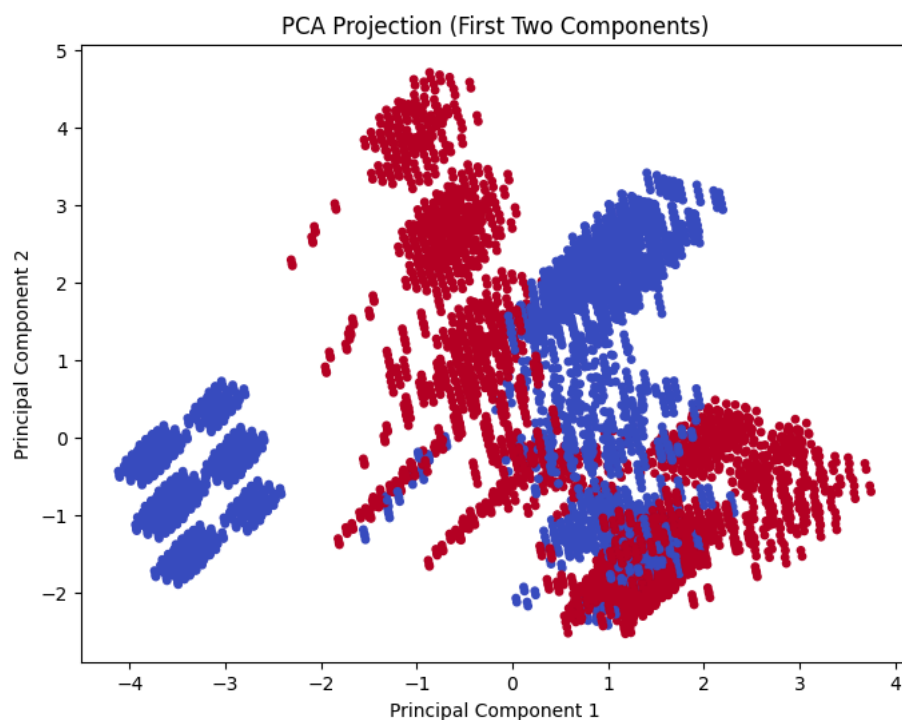
```

Reduced data shape: (8124, 15)

```

plt.figure(figsize=(8,6))
plt.scatter(X_reduced[:, 0], X_reduced[:, 1],
            c=y.map({'p': 0, 'e': 1}), cmap='coolwarm', s=15)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA Projection (First Two Components)')
plt.show()

```



```
print(f"The mushroom dataset originally had {X.shape[1]} features.")
print(f"After PCA, {k} principal components were sufficient to retain 95% of variance.")
print("This means dimensionality was reduced significantly while preserving most of the information.")
print("The scatter plot above shows that the first two components already provide visible class separation between
```

The mushroom dataset originally had 4 features.
After PCA, 15 principal components were sufficient to retain 95% of variance.
This means dimensionality was reduced significantly while preserving most of the information.
The scatter plot above shows that the first two components already provide visible class separation between edible and poisonous mushrooms.

III. Implement Random Forest Algorithm using the Bill authentication Dataset and evaluate its performance using appropriate metrics.

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00267/data_banknote_authentication.txt"
```

```
columns = ['variance', 'skewness', 'curtosis', 'entropy', 'class']
```

```
df = pd.read_csv(url, header=None, names=columns)
```

```
df.shape
```

```
(1372, 5)
```

```
df.head()
```

	variance	skewness	curtosis	entropy	class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
X = df.drop('class', axis=1)
y = df['class']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```
y_pred = rf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
```

```
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

```
Accuracy: 0.9927
Precision: 1.0000
Recall: 0.9843
F1 Score: 0.9921
```

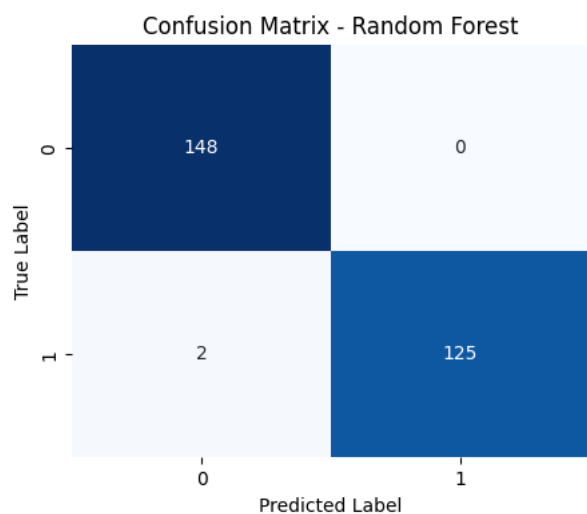
```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.99       1.00       0.99       148
     1       1.00       0.98       0.99       127

 accuracy          0.99
 macro avg         0.99
 weighted avg      0.99
```

```
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Random Forest')
plt.show()
```



IV. Implement Ada boost algorithm to classify flower species.

```
iris = datasets.load_iris()
```

```
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=0.2, random_state=42)
```

```
base = DecisionTreeClassifier(max_depth=1, random_state=42)
clf = AdaBoostClassifier(estimator=base, n_estimators=100, random_state=42)
```

```
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

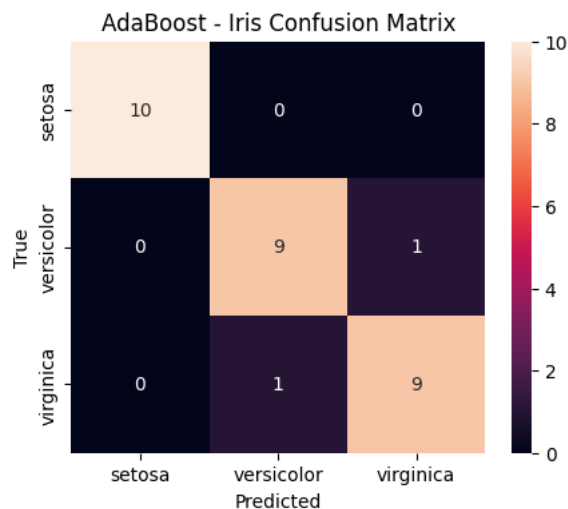
```
print('Accuracy:', accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.9333333333333333
```

```
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=iris.target_names,
yticklabels=iris.target_names)
plt.xlabel('Predicted'); plt.ylabel('True'); plt.title('AdaBoost - Iris Confusion Matrix')
plt.show()
```



```
cv = cross_val_score(clf, X, y, cv=5, scoring='accuracy')
print('5-fold CV accuracy mean:', cv.mean())
```

```
5-fold CV accuracy mean: 0.9533333333333334
```

V. Comparison of various Classification Machine Learning algorithms and evaluate its performance using the appropriate metrics.

```
iris = datasets.load_iris()
```

```
X = iris.data
y = iris.target
```

```
scaler = StandardScaler()
Xs = scaler.fit_transform(X)
```

```
models = {
'LogisticRegression': LogisticRegression(max_iter=500, random_state=42),
'SVM': SVC(probability=True, random_state=42),
'RandomForest': RandomForestClassifier(n_estimators=200, random_state=42),
'AdaBoost': AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1), n_estimators=100, random_state=42)
}
```



```
scoring = ['accuracy','precision_macro','recall_macro','f1_macro']
```

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
results = {}
```

```
for name, model in models.items():  
    scores = cross_validate(model, Xs, y, cv=cv, scoring=scoring)  
    results[name] = {m: np.mean(scores['test_' + m]) for m in ['accuracy','precision_macro','recall_macro','f1_macro']}
```

```
df_res = pd.DataFrame(results).T  
print(df_res)
```

	accuracy	precision_macro	recall_macro	f1_macro
LogisticRegression	0.953333	0.954882	0.953333	0.953216
SVM	0.953333	0.954882	0.953333	0.953216
RandomForest	0.960000	0.963300	0.960000	0.959798
AdaBoost	0.960000	0.963300	0.960000	0.959798

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.